

Αλγόριθμοι και Πολυπλοκότητα Δεύτερη Σειρά Γραπτών Ασκήσεων

Παναγή Αντρέας
Α.Μ. : 7115 1424 00008

Π.Μ.Σ. - ΑΛΜΑ

Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Εθνικό Μετσόβιο Πολυτεχνείο
Αθήνα, Ελλάδα *

Δεκέμβριος 2024



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

* Στοιχειοθέτηση με χρήση $\text{\LaTeX}$

Περιεχόμενα

Άσκηση 1: Παιχνίδι με Κάρτες	1
Εκφώνηση	1
Λύση	1
Μέθοδος κατασκευής αλγορίθμου	1
Αλγόριθμος	1
Πολυπλοκότητα	1
Ορδότητα	2
Άσκηση 2: Ταξιδεύοντας στον Κόσμο	3
Εκφώνηση	3
Λύση	3
Πρώτο Ερώτημα	3
Δεύτερο Ερώτημα	3
Άσκηση 3: Επίλυση Συστήματος Ανισοτήτων	5
Εκφώνηση	5
Λύση	5
Ερώτημα (α)	5
Ερώτημα (β)	6
Ερώτημα (γ)	7
Άσκηση 4: Μετατροπή Ροής Ακμών σε Ροή Μονοπατιών	8
Εκφώνηση	8
Λύση	8
Λύση: Θεώρημα Flow Decomposition	8
Λύση: Θεώρημα Καραθεοδωρή	9
Άσκηση 5: Επιβεβαίωση και Αναπροσαρμογή Μέγιστης Ροής	11
Εκφώνηση	11
Λύση	11
Ερώτημα 1.	11
Ερώτημα 2.	12
Ερώτημα 3. (Bonus)	12

Άσκηση 1: Παιχνίδι με Κάρτες

Εκφώνηση

Θεωρούμε το εξής παιχνίδι ανάμεσα σε δύο παίκτες, τον X και τον Y . Έχουμε μια ακολουθία n καρτών που είναι τοποθετημένες σε ένα τραπέζι, η μία δίπλα στην άλλη, σε μια νοητή ευθεία. Σε κάθε κάρτα i υπάρχει ένας δετικός ακέραιος αριθμός a_i . Οι αριθμοί όλων των καρτών είναι ορατοί και στους δύο παίκτες. Οι παίκτες παίζουν με τη σειρά, πρώτα ο X , μετά ο Y , μετά πάλι ο X , κοκ. Στη σειρά του, κάθε παίκτης παίρνει είτε την πρώτη είτε την τελευταία κάρτα από αυτές που έχουν απομείνει στο τραπέζι (το “πρώτη” και το “τελευταία” αναφέρεται στις θέσεις των καρτών στην ευθεία, ο παίκτης πρέπει υποχρεωτικά να πάρει μία από τις δύο αυτές κάρτες). Το παιχνίδι ολοκληρώνεται όταν δεν έχει απομείνει καμία κάρτα στο τραπέζι.

Στόχος των παικτών είναι να μεγιστοποιήσουν το συνολικό τους κέρδος, δηλ. το άθροισμα των αριθμών στις κάρτες που έχουν συλλέξει στο τέλος του παιχνιδιού. Θεωρούμε ότι και οι δύο παίκτες ακολουθούν βέλτιστη στρατηγική, δηλ. επιλέγουν τις κάρτες που μεγιστοποιούν το κέρδος τους στο τέλος του παιχνιδιού, δεδομένου ότι ο άλλος παίκτης ακολουθεί και αυτός βέλτιστη στρατηγική.

Να διατυπώσετε αποδοτικό αλγόριθμο που υπολογίζει το συνολικό κέρδος των δύο παικτών στο τέλος του παιχνιδιού, δεδομένου ότι και οι δύο ακολουθούν βέλτιστη στρατηγική. Να αιτιολογήσετε την ορθότητα και την υπολογιστική πολυπλοκότητα του αλγορίθμου σας.

Λύση

Μέθοδος κατασκευής αλγορίθμου

Θεωρούμε την εξής ακολουθία καρτών $a = (1, 2, 13, 2)$ και παρατηρούμε ότι αν ο X λειτουργεί με άπληστο τρόπο (επιλέγοντας πάντα την μεγαλύτερη κάρτα) τότε η στρατηγική αυτή δεν είναι βέλτιστη. Επομένως “οδηγούμαστε” σε κατασκευή αλγορίθμου με χρήση δυναμικού προγραμματισμού.

Αλγόριθμος

Έστω $a = (a_1, \dots, a_n)$ η ακολουθία καρτών.

Θεωρούμε τον πίνακα $D[i][j][t] = [(s_X, s_Y)]$, ο πίνακας αυτός έχει $2n^2 = O(n^2)$ θέσεις και έχει σε κάθε θέση το στοιχείο του αποτελείται από το ζεύγος (s_X, s_Y) τέτοιο ώστε τα s_X, s_Y αντιστοιχούν στο μέγιστο κέρδος που μπορεί να αποκομίσουν οι παίκτες X, Y αντίστοιχα, αν θεωρήσουμε το παιχνίδι περιορισμένο, μεταξύ των καρτών i και j (όπου προφανώς $i \leq j$) όπου σειρά έχει (να παίξει) ο παίκτης $t, t \in \{X, Y\}$.

Είσοδος: Ακολουθία $a = (a_1, \dots, a_n)$ και

Έξοδος: $D[1][n][X]$

1. Αρχικοποιούμε τον πίνακα:

$$(a') \quad D[i][i][X] = (a_i, 0), \forall i \in [n]$$

$$(b') \quad D[i][i][Y] = (0, a_i), \forall i \in [n]$$

2. Δίνουμε ως αρχικές τιμές τα $i = 1, j = n, t = X$

(a') Αν $i = j$ (Βάση Αναδρομής) επέστρεψε $D[i][i][t]$ (έχουν αρχικοποιηθεί)

$$(b') \quad D[i][j][t] = \max\{D[i+1][j][\{X, Y\} \setminus \{t\}] + a_i, D[i][j-1][\{X, Y\} \setminus \{t\}] + a_j\}$$

Πολυπλοκότητα

Γίνονται $2n^2$ καταχωρήσεις οι οποίες χρειάζονται $O(1)$ χρόνο. Οπότε ο συνολικός χρόνος του αλγορίθμου είναι $O(n^2)$

Ορθότητα

Θα αποδείξουμε την ορθότητα με επαγωγή στο πλήθος των καρτών (n).

Επαγωγική Βάση: Σε περίπτωση που έχουμε μια κάρτα, θα την επιλέξει ο X και θα έχουμε $(s_X = a_i, s_Y = 0)$, πράγματι, ο αλγόριθμος στο πρώτο βήμα δίνει στον πίνακα την τιμή αυτή.

Επαγωγική Υπόθεση: Έστω ότι ο αλγόριθμος δουλεύει σωστά για $n < k$, $k \in \mathbb{N}_{\geq 2}$.

Επαγωγικό Βήμα: Θα δείξουμε ότι ο αλγόριθμος δουλεύει σωστά για $n = k$ κάρτες.

Αρχικά, θα ξεκινήσει ο παίχτης X και θα διαλέξει μεταξύ των a_1 και a_n , αν γνωρίζαμε το ορθό, βέλτιστο κέρδος για το παιχνίδι περιορισμένο στις κάρτες 1 με $n - 1$ και 2 με n (επαγωγική υπόθεση), τότε θα έπρεπε να διαλέξουμε μεταξύ των δύο και μοναδικών περιπτώσεων $D[1][n][X] = \max\{D[2][n][Y] + a_1, D[1][n - 1][Y] + a_n\}$, ο αλγόριθμος συγκρίνει τις δύο ποσότητες και επιλέγει το μέγιστο.

Άσκηση 2: Ταξιδεύοντας στον Κόσμο

Εκφώνηση

Θεωρούμε μη κατευθυνόμενο γράφημα $G = (V, E, \vec{c}, \vec{\tau})$ με n κορυφές, m ακμές και θετικά ακέραια κόστη $\vec{c}$ και $\vec{\tau}$ στις ακμές. Κάθε ακμή $e = \{x, y\} \in E$ δηλώνει ότι οι πόλεις x και y συνδέονται με αμφίδρομη σιδηροδρομική σύνδεση, το κόστος $\vec{c}(e)$ δηλώνει την τιμή της ετήσιας κάρτας για απεριόριστες μετακινήσεις μεταξύ των πόλεων x και y , και το κόστος $\vec{\tau}(e)$ δηλώνει την τιμή του εισιτηρίου για απλή μετάβαση μεταξύ των πόλεων x και y .

Σχεδιάζετε την αγορά ενός συνόλου ετήσιων καρτών με ελάχιστο συνολικό κόστος, οι οποίες επιτρέπουν απεριόριστες μετακινήσεις μεταξύ της πόλης s , όπου είναι το πατρικό σας, και της πόλης t , όπου σπουδάζετε.

Η Χώρα των Αλγορίθμων έχει πυκνό σιδηροδρομικό δίκτυο. Έτσι υπάρχουν πολλά διαφορετικά τέτοια σύνολα ετήσιων καρτών με ελάχιστο συνολικό κόστος που προσφέρουν απεριόριστες μετακινήσεις μεταξύ των πόλεων s και t . Αποφασίζετε λοιπόν να αγοράσετε εκείνο το σύνολο καρτών που (εκτός από το κόστος αγοράς) ελαχιστοποιεί και το επιπλέον κόστος μετακίνησης (με εισιτήριο απλής μετάβασης) μεταξύ της πόλης a , όπου βρίσκεται η έδρα της αγαπημένης σας ομάδας, και της πόλης b , όπου πηγαίνετε συχνά για πεζοπορία και σκι. Με βάση τον κανονισμό της Εταιρείας Σιδηροδρόμων, για την μετακίνηση μεταξύ των πόλεων a και b , πληρώνετε τα εισιτήρια $\vec{\tau}(e)$ μόνο για τις σιδηροδρομικές συνδέσεις e που δεν έχετε ετήσια κάρτα απεριόριστων μετακινήσεων.

Να διατυπώσετε αποδοτικό αλγόριθμο για τον υπολογισμό ενός συντομότερου $s - t$ μονοπατιού, με βάση τα κόστη $\vec{c}$, που επιπλέον ελαχιστοποιεί το κόστος εισιτηρίων $\vec{\tau}$ για την μετακίνηση μεταξύ των πόλεων a και b . Να εξηγήσετε προσεκτικά τα ενδιάμεσα βήματα του αλγορίθμου σας, αιτιολογώντας την ορθότητά του, και να αιτιολογήσετε την υπολογιστική του πολυπλοκότητα (είναι καλό ο χρόνος εκτέλεσης του αλγορίθμου σας να μην ξεπερνάει σημαντικά τον χρόνο για τον υπολογισμό ενός συντομότερου $s - t$ μονοπατιού).

Λύση

Πρώτο Ερώτημα

Για να σχεδιάσουμε την αγορά ενός συνόλου ετήσιων καρτών με ελάχιστο συνολικό κόστος οι οποίες επιτρέπουν άπειρες μετακινήσεις μεταξύ των πόλεων s, t , (ισοδύναμα) αναζητάμε σε μη κατευθυνόμενο γράφημα, ελάχιστο μονοπάτι από την s στην t .

Η εύρεση μονοπατιού, ελαχίστου βάρους, μπορεί να γίνει με χρήση του αλγορίθμου Dijkstra, όπου στο τέλος θα επιλέξουμε τις κάρτες που αντιστοιχούν σε ακμές του ελαχίστου (s, t) -μονοπατιού που επέστρεψε ο Dijkstra.

Δεύτερο Ερώτημα

Έστω ότι απόσταση μιας κορυφής από ένα μονοπάτι ορίζουμε την μικρότερη απόσταση της κορυφής από τους κόμβους του μονοπατιού.

Θα κατασκευάσουμε το $s - t$ μονοπάτι P με τέτοιο τρόπο ώστε να είναι ελάχιστα $(s - t)$ μονοπάτι, και να έχει το ελάχιστο άθροισμα αποστάσεων από τις κορυφές a, b .

$$P = \arg \min_P \{ \text{dist}(v, a) + \text{dist}(v, b) \mid v \in P \text{ όπου } P \text{ ελάχιστο } (s, t) - \text{μονοπάτι} \}$$

Αλγόριθμος

Είσοδος: $G = (V, E, \vec{c}, \vec{\tau})$, $s, t, a, b \in V(G)$

Εξοδος: Ελάχιστο (s, t) -μονοπάτι (βάσει $\vec{c}$) που επίσης ελαχιστοποιεί την απόσταση (a, b) (βάσει $\vec{\tau}$)

1. Τρέξε Dijkstra από την κορυφή a και έστω D_a ο πίνακας με τις ελάχιστες αποστάσεις της a από τις άλλες κορυφές, βάση βάρους τ
2. Τρέξε Dijkstra από την κορυφή b και έστω D_b ο πίνακας με τις ελάχιστες αποστάσεις της b από τις άλλες κορυφές, βάση βάρους τ
- * Από την κορυφή s τρέχουμε την εξής παραλλαγή του Dijkstra:
- 3 **For** $u \in V(G) \setminus \{s\}$
 - 4 $d[u] \leftarrow \infty$
 5. $p[u] \leftarrow \text{null}$
6. $d[s] \leftarrow 0$
7. Δημιουργούμε μια ουρά προτεραιότητας Q ¹ με κλειδιά τις αποστάσεις d
8. **While** $Q \neq \emptyset$
 9. Εξήγαγε το πρώτο (ελάχιστο) στοιχείο u της Q .
 10. **for** $e = \{u, v\} \in E(G)$
 11. **If** $(d[v] > d[u] + c(e))$ **or**
 $(d[v] = d[u] + c(e) \text{ and } \text{dist}_\tau(u, a) + \text{dist}_\tau(u, b) < \text{dist}(p[v], a) + \text{dist}(p[v], b))$
 12. Ενημέρωσε το κλειδί της Q για την v σε $d[u] + c(e)$
 13. $p[v] \leftarrow u$
 - 14 Έστω P_1 το ελάχιστο $s - t$ μονοπάτι του αλγορίθμου.
Θεωρούμε το $G' = (V(G), E(G), \tau')$, όπου $\tau'(e) = 0 \forall e \in P_1$ και $\tau'(e) = \tau(e) \forall e \notin P_1$
 - 15 Τρέχουμε τώρα Dijkstra στο G' και βρίσκουμε το ελάχιστο (a, b) -μονοπάτι.

Πολυπλοκότητα

Η πολυπλοκότητα του αλγορίθμου είναι όσο η πολυπλοκότητα υλοποίησης ενός Dijkstra.

Σε πυκνά γραφήματα ($m = O(n^2)$) επιλέγουμε υλοποίηση της ουράς προτεραιότητας με πίνακα ώστε η πολυπλοκότητα να είναι $O(n^2)$

Ορθότητα

Εκτελούμε αρχικά 2 φορές τον αλγόριθμο Dijkstra ώστε να γνωρίζουμε τις αποστάσεις κάθε κορυφής από τις a και b .

Έπειτα τρέχουμε τον τροποποιημένο αλγόριθμο Dijkstra ο οποίος (από όλα τα ελάχιστα μονοπάτια) επιστρέφει το ελάχιστο (s, t) -μονοπάτι που ελαχιστοποιεί το άθροισμα της απόστασής του από τα a, b (απόσταση ενός μονοπατιού ορίσαμε ως την ελάχιστη από τις αποστάσεις των κόμβων του με την κορυφή).

Ζητείται το ελάχιστο (s, t) μονοπάτι που θα ελαχιστοποιήσει την απόσταση (a, b) , ουσιαστικά το μονοπάτι στο οποίο θα είναι όσο πιο “κοντά” οι κορυφές a, b , με άλλα λόγια το άθροισμα των αποστάσεών τους (από ένα ελάχιστο (s, t) μονοπάτι) να είναι ελάχιστο.

Αυτό το μονοπάτι επιφέρει μετακινήσεις χωρίς κόστος, οπότε ξανατρέχουμε τον αλγόριθμο Dijkstra από την a στην b ώστε να βρούμε το ελάχιστο μονοπάτι (που ενδεχομένως να χρησιμοποιεί τώρα και τις δωρεάν ακμές).

¹Ανάλογα με τον τρόπο υλοποίησης της ουράς προτεραιότητας, παίρνουμε και διαφορετικό χρόνο εκτέλεσης

Άσκηση 3: Επίλυση Συστήματος Ανισοτήτων

Εκφώνηση

Έστω $x_1, \dots, x_n$ ακέραιες μεταβλητές. Θεωρούμε ένα σύστημα S αποτελούμενο από m ανισότητες της μορφής $x_i - x_j \leq b_{ij}$, για κάποια $1 \leq i, j \leq n$, όπου τα b_{ij} είναι ακέραιοι αριθμοί. Το S είναι ικανοποιήσιμο αν υπάρχουν ακέραιες τιμές για τις μεταβλητές $x_1, \dots, x_n$ που ικανοποιούν όλες τις ανισότητες του S .

- (α) Να διατυπώσετε ένα κριτήριο για το αν το S είναι ικανοποιήσιμο (και να αποδείξετε την ορθότητα του κριτηρίου σας). Με βάση αυτό το κριτήριο, να διατυπώσετε έναν αποδοτικό αλγόριθμο που διαπιστώνει αν το S είναι ικανοποιήσιμο ή όχι. Αν το σύστημα είναι ικανοποιήσιμο, ο αλγόριθμός σας πρέπει να υπολογίζει αποδεκτές τιμές για τις μεταβλητές $x_1, \dots, x_n$. Ποια είναι η υπολογιστική πολυπλοκότητα του αλγορίθμου σας;
- (β) Να συμπληρώσετε τον αλγόριθμο του (α) ώστε αν το σύστημα S δεν είναι ικανοποιήσιμο, να υπολογίζει ένα ελάχιστο (ως προς το πλήθος ανισοτήτων) υποσύστημα S' που δεν είναι ικανοποιήσιμο. Ποια είναι η υπολογιστική πολυπλοκότητα του αλγορίθμου σας;
- (γ) Θεωρούμε ότι κάθε ανισότητα $x_i - x_j \leq b_{ij}$ συνοδεύεται από ένα θετικό ακέραιο βάρος w_{ij} . Να διατυπώσετε αλγόριθμο που αν το σύστημα S δεν είναι ικανοποιήσιμο, υπολογίζει ένα ελάχιστου συνολικού βάρους υποσύστημα S' που δεν είναι ικανοποιήσιμο. Ποια είναι η υπολογιστική πολυπλοκότητα του αλγορίθμου σας; Είναι αυτή πολυωνυμική στο μέγεθος της εισόδου;

Λύση

Ερώτημα (α)

Κριτήριο:

Θεωρούμε το σύστημα S με $x_i \in \mathbb{Z}$, όπου $i \in [n]$ για κάποιο $n \in \mathbb{N}$, για τα οποία υπάρχουν $m \in \mathbb{N}$ ανισότητες της μορφής: $x_i - x_j \leq b_{ij}$, $1 \leq i, j \leq [n]$.

Θεωρούμε το εξής (κατευθυνόμενο) γράφημα $G = (V, E, w)$, όπου:

$V := \{x_i \mid x_i \text{ μεταβλητή του συστήματος } S, i \in [n]\}$,

$E := \{(x_j, x_i) \mid b_{ij} \text{ ακέραιος αριθμός του συστήματος } S, 1 \leq i, j \leq [n]\}$,

$w : E \rightarrow \mathbb{Z}$, όπου $e = \{x_j, x_i\} \mapsto b_{ij}$ (η συνάρτηση βαρών των ακμών του G).

Το πρόβλημα τώρα ανάγεται στην εύρεση ελάχιστον μονοπατιών.

Το ισοδύναμο κριτήριο είναι το εξής:

Το S είναι ικανοποιήσιμο αν δεν υπάρχουν αρνητικοί κύκλοι.

Απόδειξη:

“ $\Rightarrow$ ” (Θα το την αντιθετοαντιστροφή)

Έστω $C \subseteq_{\text{υπ}} G$ κύκλος με ακμές τ.ω. όλες έχουν αρνητικά βάρη $w(e) = b_{ij} < 0 \forall e \in V(C)$.

Παρατηρούμε ότι

$$\sum_{e \in V(C)} w(e) = \sum_{(x_j, x_i) \in V(C)} b_{ij} < 0$$

Έστω τώρα $x_{i_1}, \dots, x_{i_k}$ οι κορυφές του κύκλου C .

Ισχύει ότι

$$x_{i_2} - x_{i_1} < b_{i_1 i_2}, x_{i_3} - x_{i_2} < b_{i_2 i_3}, \dots, x_{i_k} - x_{i_{k-1}} < b_{i_{k-1} i_k}, x_{i_{k-1}} - x_{i_k} < b_{i_k i_{k-1}}$$

Προσθέτοντας κατά μέλη προκύπτει ότι

$$0 \leq b_{i_1 i_2} + \dots + b_{i_{k-1} i_k} \implies \sum_{(x_j, x_i) \in V(C)} b_{ij} \geq 0$$

Άτοπο.

“ $\Leftarrow$ ” Θεωρούμε τώρα μια αυθαίρετη κορυφή $s \in V(G)$ και εκτελούμε τον αλγόριθμο Bellman Ford με αφετηρία την κορυφή s .

Έστω $d(x_i)$ η απόσταση ο κόμβου x_i από την s που δίνει αλγόριθμος B-F, όπου $i \in [n]$.

Θεωρούμε ότι ως τιμή κάθε μεταβλητής x_i του S την τιμή $d(x_i)$.

Αφού δεν υπάρχουν αρνητικοί κύκλοι τότε τα $d(x_i), i \in [n]$ αποτελούν πράγματι ελάχιστη απόσταση.

Για κάθε ακμή $e = \{x_j, x_i\} \in E(G)$, παρατηρούμε ότι $d(x_i) \leq d(x_j) + b_{ij}$ (το μήκος του συντομότερου (s, x_i) μονοπατιού είναι μικρότερο ή ίσο απο το μήκος του $(s, x_j) \cup (x_j, x_i)$, όπου (s, x_j) στο συντομότερο $s - x_j$ μονοπάτι).

Επομένως για τις τιμές που δέσαμε στις μεταβλητές του S θα ισχύει ότι $x_i \leq x_j + b_{ij} \iff x_i - x_j \leq b_{ij}$.

Αλγόριθμος

Όπως δείξαμε και στην απόδειξη

1. Εκτελούμε τον αλγόριθμο Bellman Ford στο γράφημα G (που εξαρτάται από το S) ξεκινώντας από μια αυθαίρετη κορυφή
2. Αποθηκεύουμε τις τιμές μετά από $n - 1$ επαναλήψεις
3. Εκτελούμε ακόμη μια επανάληψη
 - Αν οι τιμές μειωθούν τότε το S δεν ικανοποιείται
 - Αν οι τιμές δεν μειωθούν τότε το S είναι ικανοποιήσιμο και μία ανάθεση τιμών είναι οι ελάχιστες αποστάσεις που επιστρέφει ο αλγόριθμος B-F (όπως δείξαμε και στην απόδειξη)

Πολυπλοκότητα:

Πρέπει να συνυπολογίζουμε τον χρόνο κατασκευής του γραφήματος και τον χρόνο που τρέχει ο αλγόριθμος B-F σε αυτό. Ο δεύτερος είναι (συνήθως) μεγαλύτερος από τον πρώτο, οπότε ο συνολικός χρόνος του αλγορίθμου θα είναι $O(nm)$.

Ερώτημα (6)

Αν το S είναι μη ικανοποιήσιμο, το ελάχιστο υποσύστημα ως προς πλήθος ανισοτήτων το οποίο είναι μη ικανοποιήσιμο, αντιστοιχεί στον αρνητικό κύκλο του γραφήματος G με τις λιγότερες ακμές και αντίστροφα.

Οι κόμβοι για τους οποίους η απόσταση από την κορυφή s μειώνεται μετά το πέρας των $n - 1$ επαναλήψεων του Bellman-Ford, αποτελούν κορυφές ενός αρνητικού κύκλου.

Αλγόριθμος:

Θα εξετάσουμε κάθε ακμή αν ανήκει σε ελάχιστο κύκλο.

1. Κατασκευάζουμε το $H \subseteq_{v\pi} G$ που προκύπτει από τις κορυφές και τις ακμές που ανήκουν σε αρνητικούς κύκλους (συνέχεια προηγούμενου αλγορίθμου).

Μπορούμε να τις βρούμε ως εξής: Εκτελώντας μια επιπλέον επανάληψη ($|V|$ -οστή) και αν κάποια κορυφή μέσω μιας ακμής μπορεί να ξανά μειωθεί, τότε η κορυφή και η ακμή ανήκουν σε αρνητικό κύκλο.

Αρκεί τώρα να βρούμε ένα αρνητικό κύκλο με ελάχιστο πλήθος ακμών στο H .

2. Για κάθε κορυφή $v \in V(H)$

(α') Τρέχουμε BFS και βρίσκουμε τον ελάχιστο (ως προς πλήθος ακμών) αρνητικό κύκλο που περιέχει την u (ο συντομότερος αρνητικός κύκλος που περιέχει την v έχει μήκος $2k - 1$ όπου k το πρώτο επίπεδο που περιέχει κορυφή ώστε 2 κόμβοι του προηγούμενου επιπέδου να καταλήγουν σε αυτήν και να έχει αρνητικό άθροισμα βαρών)

3. Κρατάμε τον ελάχιστο από τους κύκλους που βρήκαμε και σημειώνουμε τις κορυφές και ακμές (και ακμές) που τον αποτελούν.
4. Βρίσκουμε τις ανισότητες που αντιστοιχούν στις ακμές (και κορυφές) του ελάχιστου κύκλου που βρήκαμε

Πολυπλοκότητα

Ο αλγόριθμος χρειάζεται $O(|V| \cdot (|V| + |E|))$ χρόνο λόγω του βήματος 2 (η κατασκευή του H δεν βοηθάει σε ανάλυση χειρότερης περίπτωσης).

Ερώτημα (γ)

Ισοδύναμα με την εύρεση μη ικανοποιήσιμου υποσυστήματος με μέγιστο βάρος, αποτελεί η εύρεση κύκλου με αρνητικό συνολικό άθροισμα από βάρη b_{ij} και μέγιστο άθροισμα από βάρη w_{ij} .

Το πρόβλημα είναι NP πλήρες, το οποίο μπορεί ναδειχθεί κατασκευάζοντας πολυωνυμικό επαληθευτή (για την απόδειξη ότι ανήκει στο NP) και με αναγωγή από το πρόβλημα ΚΥΚΛΟΣ ΧΑΜΙΛΤΟΝ. Επομένως δεν υπάρχει (ακόμη) πολυωνυμικός (ντετερμινιστικός) αλγόριθμος.

Άσκηση 4: Μετατροπή Ροής Ακμών σε Ροή Μονοπατιών

Εκφώνηση

Θεωρούμε $s - t$ δίκτυο $G = (V, E, \vec{c})$ με n κορυφές, m (κατευθυνόμενες) ακμές, και θετική ακέραια χωρητικότητα $\vec{c}(e)$ σε κάθε ακμή $e \in E$. Δίνεται μια (ακέραια) $s - t$ ροή ακμών $f : E \rightarrow \mathbb{N}$ που αναθέτει ακέραια ροή $f(e)$, με $0 \leq f(e) \leq \vec{c}(e)$, σε κάθε ακμή $e \in E$ του G (και εξασφαλίζει διατήρηση ροής σε κάθε ενδιάμεση κορυφή $v \in V \setminus \{s, t\}$). Θα λέμε ότι μια $s - t$ ροή ακμών f είναι ακυκλική όταν δεν υπάρχει κύκλος c στο G τέτοιος ώστε $f(e) > 0$ για κάθε ακμή $e \in c$.

Έστω $\mathcal{P}$ το σύνολο όλων των $s - t$ μονοπατιών στο G . Μια (ακέραια) ροή μονοπατιών $g : \mathcal{P} \rightarrow \mathbb{N}$ αναθέτει ακέραια ροή $g(p)$ σε κάθε $s - t$ μονοπάτι $p \in \mathcal{P}$, έτσι ώστε $\sum_{p \in \mathcal{P} : e \in p} g(p) \leq \vec{c}(e)$, για κάθε ακμή $e \in E$ (η διατήρηση ροής στις ενδιάμεσες κορυφές εξασφαλίζεται τετριμμένα, αφού η g είναι ροή $s - t$ μονοπατιών).

Λέμε ότι μια $s - t$ ροή ακμών $f : E \rightarrow \mathbb{N}$ αντιστοιχεί σε μια $s - t$ ροή μονοπατιών $g : \mathcal{P} \rightarrow \mathbb{N}$, αν $f(e) = \sum_{p \in \mathcal{P} : e \in p} g(p)$ για κάθε ακμή $e \in E$. Να διατυπώσετε αποδοτικό αλγόριθμο ο οποίος με είσοδο μια ακυκλική $s - t$ ροή ακμών $f : E \rightarrow \mathbb{N}$, υπολογίζει μια αντίστοιχη $s - t$ ροή μονοπατιών $g : \mathcal{P} \rightarrow \mathbb{N}$ τέτοια ώστε $g(p) > 0$ για m το πολύ μονοπάτια $p \in \mathcal{P}$. Πως εξασφαλίζει ο αλγόριθμός σας τον τελευταίο περιορισμό και ποια είναι η υπολογιστική του πολυπλοκότητα;

Λύση

Λύση: Θεώρημα Flow Decomposition

Θεώρημα: (Flow Decomposition)

Έστω $(G = (V, E), s, t, c)$ ένα δίκτυο και f μια ροή στο δίκτυο.

Τότε υπάρχει συλλογή ροών $f_1, \dots, f_k$ και συλλογή $s \rightarrow t$ μονοπατιών τέτοια ώστε $p_1, \dots, p_k$ τέτοια ώστε:

1. $k \leq |E|$
2. Το κόστος της ροής f είναι ίσο με το άροισμα από τα κόστη των f_i
3. Τα κόστη f_i στέλνουν θετική ροή μόνος στις ακμές των p_i

Απόδειξη:

Η απόδειξη είναι κατασκευαστική. Ο αλγόριθμος που ακολουθεί κατασκευάζει τις συλλογές ροών και μονοπατιών που ικανοποιούν τις παραπάνω ιδιότητες.

1. $i := 1$
2. $r := f$
3. **While** μέγεθος(r) > 0 ($r \neq 0$)
 4. Βρες p_i , όπου το p_i είναι ένα $s - t$ μονοπάτι με ακμές $(u, v) : r(u, v) > 0$
 5. Θέσε $f_{\min} := \min\{r(u, v) \mid (u, v) \in p_i\}$
 6. Θέσε $f_i(u, v) = f_{\min} \forall (u, v) \in p_i$ και $f_i = 0 \forall (u, v) \notin p_i$
 7. Ενημέρωσε $r(u, v) \leftarrow r(u, v) - f_i(u, v)$
 8. $i \leftarrow i + 1$

Η ροή r αρχικοποιείται ως ίση με την f και σε κάθε βήμα $i = 1, \dots, k$ αν η r έχει θετικό μέγεθος (δεν είναι ταυτοτικά μηδέν) βρίσκουμε ένα μονοπάτι p_i από την $s - t$ με θετική ροή.

Αυτό το μονοπάτι πρέπει να υπάρχει, γιατί αν υποθέσουμε (ως προς άτοπο ότι δεν υπάρχει), τότε θεωρούμε A το σύνολο των κόμβων που είναι προσβάσιμοι από την s με μονοπάτια ακμών όπου

$r(u, v) > 0$. Τότε το A περιέχει την s και όχι την t και τότε αυτό θεωρείται μια αποκοπή (cut) όπου η χωρητικότητα του είναι ίση με το μέγεθος της r . Παρατηρούμε ότι αφού δεν εξέρχεται ροή από το A τότε το κόστος της ροής r θα έπρεπε να είναι μηδέν, άτοπο.

Ορίζουμε την ροή f_i στέλνοντας κατά μήκος του p_i την ελάχιστη από τις ροές που ορίζει η r στις ακμές του p_i .

Παρατηρούμε ότι οι f_i είναι πράγματι ροές γιατί, για κάθε ακμή ισχύει $f_i(u, v) \leq r(u, v)$ και απο κατασκευή ισχύει ότι $r(u, v) \leq f(u, v)$, όπου για την f ισχύει ότι $f(u, v) \leq c(u, v)$.

Στο τέλος μειώνουμε την ροή $r(u, v)$ κατά $f_i(u, v)$ για κάθε ακμή (u, v) . Τώρα παρατηρούμε επίσης ότι η r παραμένει ροή γιατί θα εξακολουθεί να ορίζει μη αρνητική ροή για κάθε ακμή (όπου επίσης συνεχίζει να ισχύει η διατήρηση της ροής).

Μετά την ενημέρωση των ροών της r , το μέγεθος της μειώνεται κατά το μέγεθος της f_i , επομένως μετά το πέρας $l \in [k]$ βημάτων θα ισχύει ότι:

$$\text{μέγεθος}(f) = \text{μέγεθος}(r) + \text{μέγεθος}(f_1) + \dots + \text{μέγεθος}(f_l).$$

Μένει να παρατηρήσουμε ότι, μετά το πέρας της ενημέρωσης του r τουλάχιστον μια ακμή η οποία είχε $r(u, v) > 0$ τώρα έχει $r(u, v) = 0$ (λόγω της υπάρξης μιας ακμής που δίνει το ελάχιστη ροή στο p_i).

Αυτό συνεπάγεται ότι μετά το πέρας l βημάτων ο αριθμός των ακμών (u, v) τέτοιων ώστε $r(u, v) > 0$ θα είναι το πολύ $|E| - l$ και συγκεκριμένα ο αλγόριθμος θα τερματίσει μετά το πέρας $|E|$ επαναλήψεων.

Θέτουμε k το πλήθος των επαναλήψεων μέχρι την (βέβαιη) περάτωση του αλγορίθμου.

Όταν θα έχει τελειώσει ο αλγόριθμος θα ισχύει ότι $\text{μέγεθος}(r) = 0$ και $\text{μέγεθος}(f) = \text{μέγεθος}(f_1) + \dots + \text{μέγεθος}(f_k)$ και τα μονοπάτια καθώς και οι ροές που βρήκαμε ικανοποιούν τις συνθήκες που θέσαμε στην αρχή.

Προφανώς, ορίζουμε $g : \mathcal{P} \rightarrow \mathbb{N}$ όπου $p_i \mapsto f_i(e)$, $e \in p_i$ ($\forall e \in p_i$, η $f_i(e)$ παραμένει σταθερή).

Υπολογιστική Πολυπλοκότητα:

Θα εκτελεστούν το πολύ $|E|$ επαναλήψεις του βρόγχου *while*.

Κάθε επανάληψη χρειάζεται $O(|V| + |E|)$ λόγω της αναζήτησης στο γράφημα (για απαντητικό μονοπάτι και ενημέρωσης).

Επομένως συνολικά χρειάζεται $O(|E| \cdot (|V| + |E|))$.

Λύση: Θεώρημα Καραθεοδωρή

Έστω $m = |E|$ και $E = \{e_1, \dots, e_m\}$.

Θεωρούμε τώρα την κάθε ροή ως $f = (p_1, \dots, p_m) \in \mathbb{Z}^m \subseteq \mathbb{R}^m$ όπου η κάθε συντεταγμένη f_i , $i \in [m]$ αντιστοιχεί στην ροή της ακμής e_i .

Θεωρούμε $\mathcal{F} := \{f \in \mathbb{Z}^m \subseteq \mathbb{R}^m \mid f \text{ ακυκλική ροή}\}$.

Από Θεώρημα Καραθεοδωρή, για κάθε $f \in \text{Cone}(\mathcal{F})$ υπάρχουν το πολύ m το πλήθος $f^1, \dots, f^m \in \mathcal{F}$ ακρότατα, γραμμικώς ανεξάρτητα, ώστε $f = f^1 + \dots, f^m$.

Αφού $f^1, \dots, f^m \in \mathcal{F}$ τότε αποτελούν ροές και είναι γραμμικώς ανεξάρτητα και αποτελούν ακρότατα του συνόλου $\text{Con}(\mathcal{F})$, τότε αναγκαστικά θα πρέπει να έχουν σταθερή ροή σε ένα μονοπάτι του γραφήματος.

Έστω $p^1, \dots, p^n$ τα (s, t) μονοπάτια που αντιστοιχούν στις $f^1, \dots, f^m$ και έστω $v(f^i)$, η (σταθερή) δεκτική ροή που αναδέτουν στο αντίστοιχο μονοπάτι (στις υπόλοιπες ακμές αναδέτουν μηδενική ροή).

θέτουμε $g : \mathcal{P} := \{p \mid p = (s - t) \text{ μονοπάτι}\} \rightarrow \mathbb{N}$ τ.ω

$$g(p) = \begin{cases} v(f^i) & , \text{αν } p = p^i, \ i \in [m] \\ 0 & , \text{αλλιώς} \end{cases}$$

Άσκηση 5: Επιβεβαίωση και Αναπροσαρμογή Μέγιστης Ροής

Εκφώνηση

Θεωρούμε $s - t$ δίκτυο $G = (V, E, \vec{c})$ με n κορυφές, m ακμές, και θετική ακέραια χωρητικότητα $c(e)$ σε κάθε ακμή $e \in E$. Σε καθένα από τα παρακάτω ερωτήματα, να αιτιολογήσετε την ορθότητα και την υπολογιστική πολυπλοκότητα των αλγορίθμων σας.

1. Δίνεται ροή f που (υποτίθεται ότι) αποτελεί μέγιστη $s - t$ ροή στο G . Να διατυπώσετε αλγόριθμο γραμμικού χρόνου που ελέγχει αν η f αποτελεί πράγματι μια μέγιστη $s - t$ ροή στο G .
2. Έστω ότι διαπιστώνουμε ότι η f αποτελεί πράγματι μια μέγιστη $s - t$ ροή στο G , αλλά στη συνέχεια ανακαλύπτουμε ότι η πραγματική χωρητικότητα $c'(e)$ μια ακμής e είναι μικρότερη κατά $k \geq 1$ μονάδες από τη χωρητικότητα $c(e)$ που είχαμε θεωρήσει αρχικά. Να διατυπώσετε αποδοτικό αλγόριθμο που (αν χρειάζεται) τροποποιεί την f σε μία μέγιστη $s - t$ ροή f' για το δίκτυο G' , το οποίο προκύπτει από το G θέτοντας $c'(e) = c(e) - k$. Ο χρόνος εκτέλεσης του αλγορίθμου σας πρέπει να είναι σημαντικά μικρότερος από τον χρόνο υπολογισμού της f' από την αρχή.
3. **(Bonus)** Λόγω μιας φυσικής καταστροφής, πρέπει να διακόψουμε προσωρινά τη λειτουργία του δικτύου. Επειδή όμως η πλήρης διακοπή της ροής από το s στο t θα προκαλούσε την καταστροφή των αγωγών / ακμών του δικτύου, χρειάζεται να διατηρήσουμε μια ελάχιστη ροή $\vec{\ell}(e)$ σε κάθε ακμή e . Θέλουμε λοιπόν να υπολογίσουμε την ελάχιστη ροή g για την οποία ισχύει ότι $\vec{\ell}(e) \leq g(e) \leq \vec{c}(e)$ σε κάθε ακμή e . Να διατυπώσετε αποδοτικό αλγόριθμο που με είσοδο ένα $s - t$ δίκτυο $G = (V, E, \vec{c}, \vec{\ell})$, όπου $\vec{c}(e) \in \mathbb{N}$ είναι η μέγιστη και $\vec{\ell}(e) \in \mathbb{N}$ είναι η ελάχιστη ροή που επιτρέπουμε σε κάθε ακμή, υπολογίζει μια ελάχιστη $s - t$ ροή g .
Αν σας διευκολύνει, μπορείτε να θεωρήσετε ως δεδομένη τη μέγιστη ροή f στο αρχικό δίκτυο $G = (V, E, \vec{c})$ και ότι $f(e) \geq \vec{\ell}(e)$ σε κάθε ακμή $e \in E$. Πως αλλάζει ο ορισμός του υπολειμματικού δικτύου και ποιος είναι τώρα ο ρόλος των “επαυξητικών” μονοπατιών;
Να προσπαθήσετε να βελτιστοποιήσετε την υπολογιστική πολυπλοκότητα του αλγορίθμου σας. Να διατυπώσετε συνοπτικά το επιχείρημα που εξασφαλίζει ότι ο αλγόριθμός σας υπολογίζει πράγματι μια ελάχιστη $s - t$ ροή που σέβεται τους περιορισμούς χωρητικότητας.

Λύση

Ερώτημα 1.

Αρχικά μπορούμε να επαληθεύσουμε ότι η f αποτελεί πράγματι ροή σε γραμμικό χρόνο (ως προς το μήκος της εισόδου $|E|$).

Πράγματι, μπορούμε να δημιουργήσουμε ένα πίνακα ο οποίος, για κάθε κορυφή, θα κρατάει την ροή που εισέρχεται και εξέρχεται.

Διατρέχοντας τις (κατευθυνόμενες) ακμές $e = (u, v)$ και αν $f(e) \leq c(e)$, αυξάνουμε την τιμή της ροής που εξέρχεται από την κορυφή u και την τιμή της ροής που εισέρχεται στην v κατά $f(e)$.

Στο τέλος ελέγχουμε αν η ροή που εισέρχεται σε κάθε κορυφή ισούται με την ροή που εξέρχεται από αυτή (εκτός από τις s, t).

Αλγόριθμος

Είσοδος: Συνάρτηση $f : E \rightarrow \mathbb{N}$ που αντιστοιχεί σε ροή για το γράφημα $G = (V, E, \vec{c})$.

Εξοδος: Ναι αν είναι μέγιστη, όχι δεν είναι μέγιστη.

1. Δημιούργησε το G_f , το υπολειπόμενο γράφημα.
2. Χρησιμοποιώντας αλγόριθμο αναζήτησης στο γράφημα, έλεγξε αν υπάρχει επαυξητικό μονοπάτι
 - Αν υπάρχει, επέστρεψε ΟΧΙ
 - Αν δεν υπάρχει, επέστρεψε ΝΑΙ

Πολυπλοκότητα

Ο χρόνος που χρειάζεται ο αλγόριθμος είναι $O(n + m) + O(n + m) = 2O(n + m) = O(n + m)$, πράγματι γραμμικός ως προς το μέγεθος της εισόδου $O(m)$ (η συνάρτηση είναι ορισμένη στις ακμές).

Ερώτημα 2.

Αλγόριθμος

1. Αν $c'(e) > f(e)$ τότε η ροή εξακολουθεί να είναι μέγιστη. Τερμάτισε.
2. Διόρθωσε την ροή f ώστε να είναι εφικτή ροή f' :
 - (α') Για την ροή της ακμής $e = (u, v)$, δέσε $f(e) = c'(e)$, υπολόγισε $k \leftarrow c(e) - c'(e)$
 - (β') Όσο το $k > 0$, επέλεξε ένα γείτονα v' της v και μείωσε την ροή του κατά $\min\{f(v, v'), d\}$, ενημέρωσε $d \leftarrow d - \min\{f(v, v'), d\}$.
 - (γ') Για όσους γείτονες μειώθηκε η ροή, επανέλαβε την διαδικασία (που εκτελέσαμε για την v)
3. Κατασκεύασε το $G_{f'}$
4. Τρέξε τον αλγόριθμο F-F στο $G_{f'}$

Πολυπλοκότητα

Θεωρούμε $n = |V|$ και $m = |E|$ και $k = c(e) - c'(e)$.

Το βήμα 1, γίνεται σε σταθερό χρόνο, το βήμα 2 με ένα *BFS*, το βήμα 3 σε $O(n + m)$ και το βήμα 4 σε $O(k(n + m))$ ή με την βελτιωμένη του μορφή $O((n_m) \log k)$.

Οπότε ο συνολικός χρόνος θα είναι $O(m \log k)$.

Ερώτημα 3. (Bonus)

Έστω ότι έχουμε ένα γράφημα $G = (V, E, \vec{c}, \vec{l})$, όπου $\vec{c} : E \rightarrow \mathbb{N}$ οι χωρητικότητες, $\vec{l} : E \rightarrow \mathbb{N}$ η ελάχιστη ροή που πρέπει να περνάει από μια ακμή και $f : E \rightarrow \mathbb{N}$ μια μέγιστη ροή και G_f το τελικό υπολειμματικό γράφημα.

Αλγόριθμος:

1. Θεωρούμε G το αρχικό γράφημα και G_f (το τελικό υπολειπόμενο γράφημα όταν βρεθεί η μέγιστη ροή)
2. Θέτουμε στο G_f , $l' : E^R(G_f) \subseteq E(G_f) \rightarrow \mathbb{N}$ τ.ω. $l'(e^R) = l(e)$ για κάθε $e \in E(G)$ (δηλαδή θέτουμε τους ελάχιστους περιορισμούς που δίνει η εκφώνηση στις (ανάποδες) ακμές του υπολημματικού γραφήματος).
3. Βρίσκουμε $t - s$ μονοπάτι p στο G_f τέτοιο ώστε $f(e) > l'(e) \forall e \in P \cap E^R(G)$ (βρίσκουμε $t - s$ μονοπάτι στο οποίο υπάρχουν ακμές οι οποίες έχουν περιθώριο να μειωθούν).
4. Ενημερώνουμε το flow του γραφήματος G , ώστε αν p το μονοπάτι και $\delta := \min\{f(e) - l'(e) > 0 | e \in E^R(G) \cap P\}$:
5. **For** $e \in P$
 6. **If** $e \in E(G)$
 7. $f(e) \leftarrow f(e) + \delta$
 8. **else**
 9. $f(e) \leftarrow f(e) - \delta$

Το υπολειπόμενο γράφημα έχει θα χρησιμοποιείται για αναζήτηση “επαυξητικών” μονοπατιών, που τώρα θα αναιρούν ροή

Τα νέα “επαυξητικά” μονοπάτια είναι αυτά που αναιρούν (παίρνουν “πίσω”) ροή από την μέγιστη.

Ορθότητα:

Ο αλγόριθμος που γράψαμε χρησιμοποιεί την αντίστροφη λειτουργία του FF, μόνο που δεν σταματάει όταν αναιρέσει όλη την ροή, αλλά όταν δεν μπορούμε να αναιρέσουμε άλλη ροή “χαλώντας” (τουλάχιστον) ένα από τους περιορισμούς για το κάτω φράγμα της ροής (κάθε ακμής).

Οπότε, ξεκινάμε από μια βέλτιστη ροή και αφαιρούμε με (αντίστοιχο τρόπο) ροή αφού βρίσκουμε “επαυξητικά” μονοπάτια από την t προς την s μέχρι να μην μπορεί να συνεχίσει η διαδικασία άλλο (κάτι που συμβαίνει μόνο όταν δεν υπάρχει μονοπάτι τέτοιο ώστε να περιέχει ακμή που μπορεί να της αφαιρεθεί (κι άλλη) ροή).

Πολυπλοκότητα:

Ο αλγόριθμος έχει πολυπλοκότητα στην χειρότερη περίπτωση όσο η πολυπλοκότητα του (σύνθετος) αλγορίθμου Ford Fulkerson.